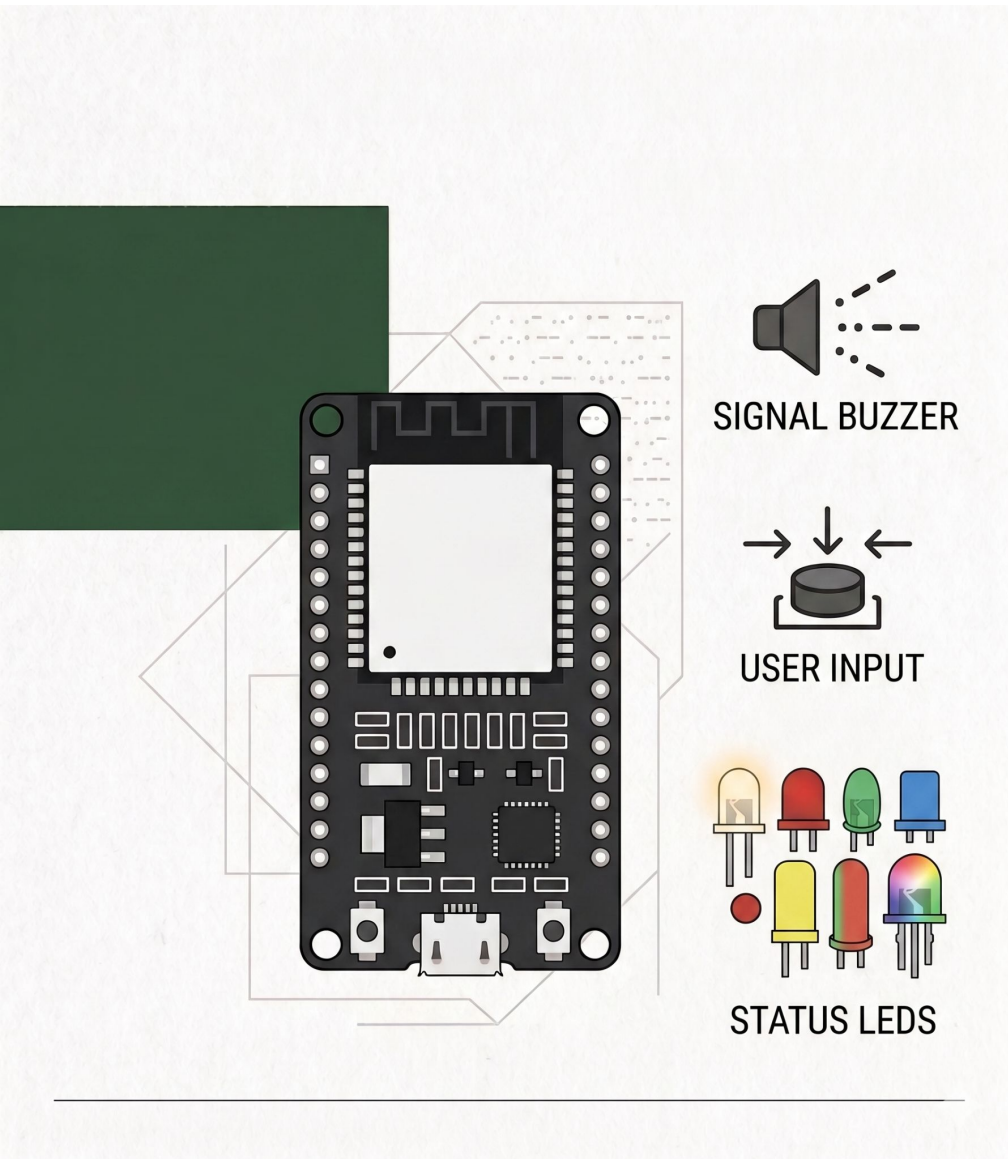


MORSE ALARM



Università degli Studi di Brescia
DIPARTIMENTO DI INGEGNERIA DELL'INFORMAZIONE
Corso di Laurea in Ingegneria Informatica

Giovanni Prossomariti



**UNIVERSITÀ
DEGLI STUDI
DI BRESCIA**

INDICE

1. Obiettivi chiave

1.1 Obiettivi didattici e tecnici

1.2 Obiettivi funzionali (User Experience)

2. Componenti e Librerie

2.1 Componenti Hardware

2.1.1 Microcontrollore: ESP32 DevKit V1

2.1.2 Output Acustico: Buzzer Piezoelettrico

2.1.3 Feedback Visivo: LED e Resistenze

2.1.4 Input Utente: Pulsanti

2.1.5 Componenti Ausiliari

2.2 Librerie Software

3. Schema dei Collegamenti

3.1 Architettura delle Connessioni

3.1.1 Tabella di Cablaggio (Pinout)

3.2 Note sulla Realizzazione

4. Logica di Controllo

4.1 Definizione degli stati del sistema

4.2 Gestione Asincrona e Debouncing

5. Analisi Energetica e Stima dei Consumi

5.1 Assorbimento dei singoli componenti

5.2 Conclusioni sulla sostenibilità energetica

6. Sviluppi e Implementazioni Future

1. Obiettivi Chiave

L'obiettivo principale del progetto **MORSE ALARM** è la progettazione e la realizzazione di un prototipo di sveglia "attiva" basata sul microcontrollore **ESP32**.

A differenza dei sistemi di allarme tradizionali, che possono essere disattivati in modo quasi inconscio, questo dispositivo mira a indurre un risveglio cognitivo immediato attraverso l'interazione con il codice Morse.

1.1 Obiettivi Didattici e Tecnici

- **Integrazione Hardware-Software:** Dimostrare la capacità di interfacciare sensori (pulsanti), attuatori (buzzer e LED) e un microcontrollore avanzato attraverso la programmazione in ambiente Arduino IDE.
- **Gestione della Connettività IoT:** Sfruttare il modulo Wi-Fi integrato nell'ESP32 per implementare il protocollo **NTP (Network Time Protocol)**, garantendo che l'orario della sveglia sia sempre sincronizzato con i server atomici globali, eliminando la necessità di moduli RTC (Real Time Clock) esterni.
- **Logica di Controllo:** Sviluppare un algoritmo capace di generare stringhe casuali e di confrontare in tempo reale l'input ricevuto dai pulsanti con la soluzione corretta.

1.2 Obiettivi Funzionali (User Experience)

- **Risveglio Attivo:** Obbligare l'utente a risolvere una sfida (la decodifica e l'inserimento di un segnale Morse) per interrompere il segnale acustico, contrastando il fenomeno dell'inerzia del sonno.
 - **Feedback Multimodale:** Fornire all'utente una risposta immediata del sistema attraverso segnali acustici (buzzer) e visivi (LED di stato), differenziando chiaramente tra inserimento corretto, errore di battuta e attivazione dell'allarme.
 - **Personalizzazione:** Creare un'interfaccia di configurazione (tramite Web Server integrato nell'ESP32) che permetta di impostare l'orario della sveglia in modo semplice da qualsiasi dispositivo collegato alla stessa rete Wi-Fi.

2. Componenti e Librerie

In questa sezione vengono descritti dettagliatamente i componenti hardware utilizzati per la realizzazione del prototipo **MORSE ALARM** e le librerie software necessarie per il controllo dei dispositivi e la connettività IoT.

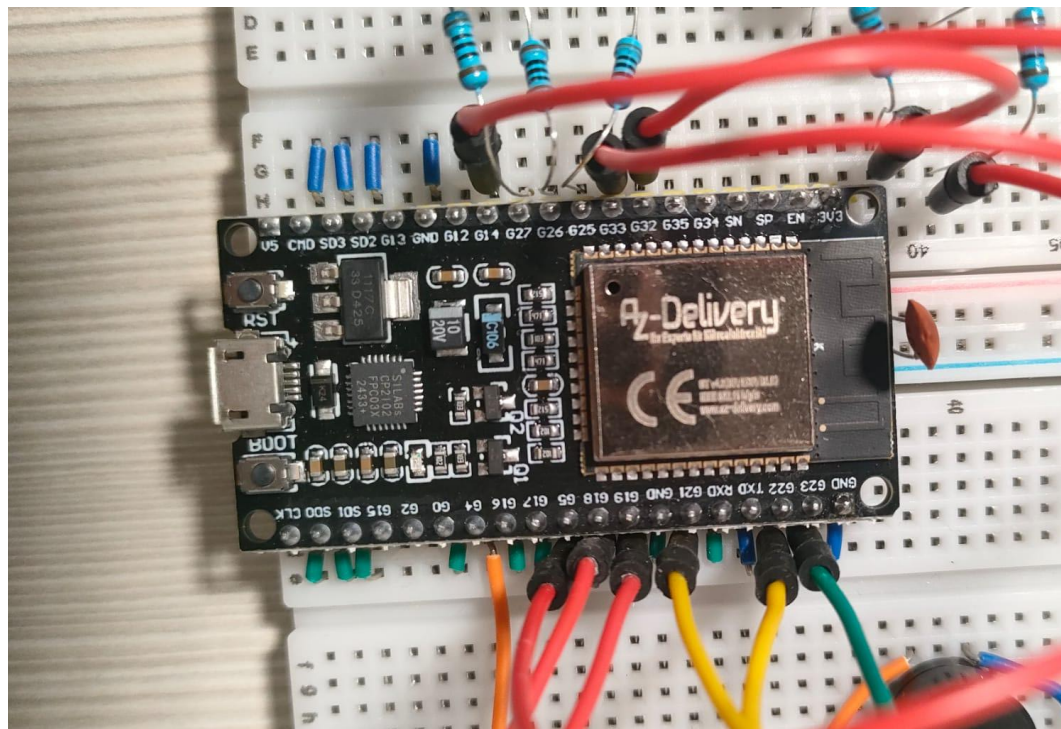
2.1 Componenti Hardware

La scelta dell'hardware è stata guidata dalla necessità di coniugare potenza di calcolo, connettività nativa e un'interfaccia utente multimodale (acustica e visiva), mantenendo al contempo un ingombro ridotto e un basso consumo energetico.

2.1.1 Microcontrollore: ESP32 DevKit V1

Il cuore del sistema è il microcontrollore **ESP32** (nello specifico, la scheda di sviluppo DevKit V1). Questa scelta è stata preferita ad altre soluzioni (come Arduino Uno o Nano) per i seguenti motivi tecnici:

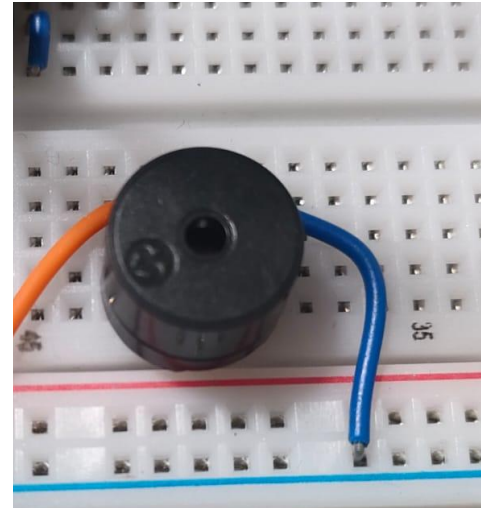
- **Connettività Nativa:** Integra moduli Wi-Fi (802.11 b/g/n) e Bluetooth (Classic e BLE), fondamentali per la sincronizzazione oraria NTP e per future espansioni IoT.
- **Prestazioni Elevate:** Basato su un processore dual-core a 32 bit (Xtensa LX6), offre la potenza necessaria per gestire contemporaneamente la connessione di rete, la logica della sfida Morse e la generazione dei segnali audio/visivi.
- **Periferiche Avanzate:** Dispone di un elevato numero di pin GPIO, DAC (Digital-to-Analog Converter) per un controllo audio più preciso e interfacce di comunicazione standard.



2.1.2 Output Acustico: Buzzer Piezoelettrico

Per l'emissione del segnale di allarme e dei segnali acustici del codice Morse, è stato utilizzato un **buzzer piezoelettrico passivo**. È controllato tramite un segnale PWM (Pulse Width Modulation) generato dall'ESP32, che permette di modulare la frequenza e creare i "toni" necessari per distinguere i punti dalle linee del codice.

È stato preferito un buzzer passivo per permettere la modulazione della frequenza tramite PWM. Mentre un buzzer attivo produce un tono fisso, il modello passivo consente di generare frequenze variabili, migliorando l'esperienza utente nella distinzione dei segnali Morse.



2.1.3 Feedback Visivo: LED e Resistenze

Il sistema di feedback visivo si basa sull'utilizzo di LED (Light Emitting Diodes):

- **LED Bianchi (x3):** Attivati ad alta luminosità per segnalare lo stato di "sequenza corretta".
- **LED Rossi (x3):** Utilizzati per segnalare la sequenza di codice morse da decifrare.
- **LED Multicolore (x1):** Un LED RGB utilizzato per indicare stati generali (connessione Wi-Fi, attesa input).

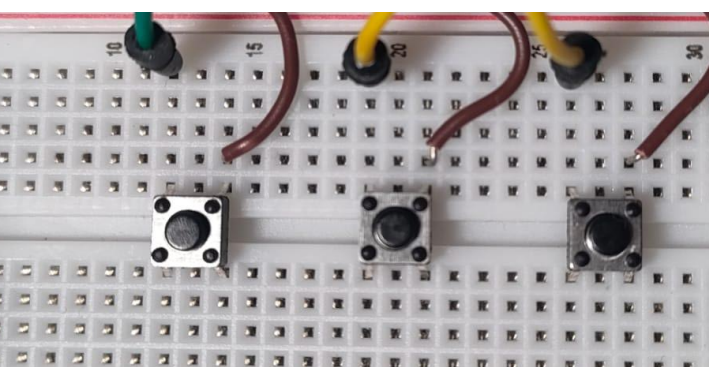
Ciascun LED è pilotato tramite una **resistenza di limitazione da 100 Ohm**

2.1.4 Input Utente: Pulsanti

Per permettere all'utente di interagire con il sistema, sono stati utilizzati tre **pulsanti a pressione (tactile switches)**, configurati come:

1. **Input "Punto":** Per l'inserimento del segnale corto.
2. **Input "Linea":** Per l'inserimento del segnale lungo.
3. **Invio (Confirm):** Per validare la sequenza inserita.

I pulsanti sono collegati in configurazione "pull-up" (sfruttando le resistenze interne dell'ESP32) per garantire letture logiche stabili.



2.1.5 Componenti Ausiliari

- **Breadboard** : Utilizzata come base sperimentale per il montaggio rapido e sicuro del circuito senza necessità di saldature.
- **Cavi Jumper** : Per le connessioni tra i componenti e la scheda ESP32.
- **Alimentazione** : Il prototipo è alimentato tramite la porta USB del computer durante la fase di sviluppo, ma può essere alimentato da un alimentatore USB standard o una batteria.



2.2 Librerie Software

Lo sviluppo del firmware è avvenuto nell'ambiente **Arduino IDE**, sfruttando la ricca ecosistema di librerie disponibili per l'ESP32. Le librerie chiave utilizzate sono:

1. **WiFi.h**: Libreria core per l'ESP32 che permette la gestione completa della connessione alla rete Wi-Fi domestica.
2. **time.h** : Utilizzata per gestire le strutture temporali e implementare il protocollo NTP. Permette di ottenere l'ora esatta dai server orari globali e di calcolare l'attivazione della sveglia.
3. **WebServer.h** : Permette di creare una interfaccia web sull'ESP32 per impostare l'orario della sveglia da smartphone.

3. Schema dei Collegamenti

In questa sezione viene illustrato il cablaggio elettrico del sistema **MORSE ALARM**. Il circuito è stato progettato per bilanciare l'uso dei pin GPIO dell'ESP32, evitando interferenze tra i segnali PWM del buzzer e i segnali digitali dei LED e dei pulsanti.

3.1 Architettura delle Connessioni

Tutti i componenti condividono una linea di massa comune (**GND**) collegata a uno dei pin GND dell'ESP32. L'alimentazione per i sensori e gli attuatori è fornita direttamente dal pin **3.3V** della scheda.

3.1.1 Tabella di Cablaggio (Pinout)

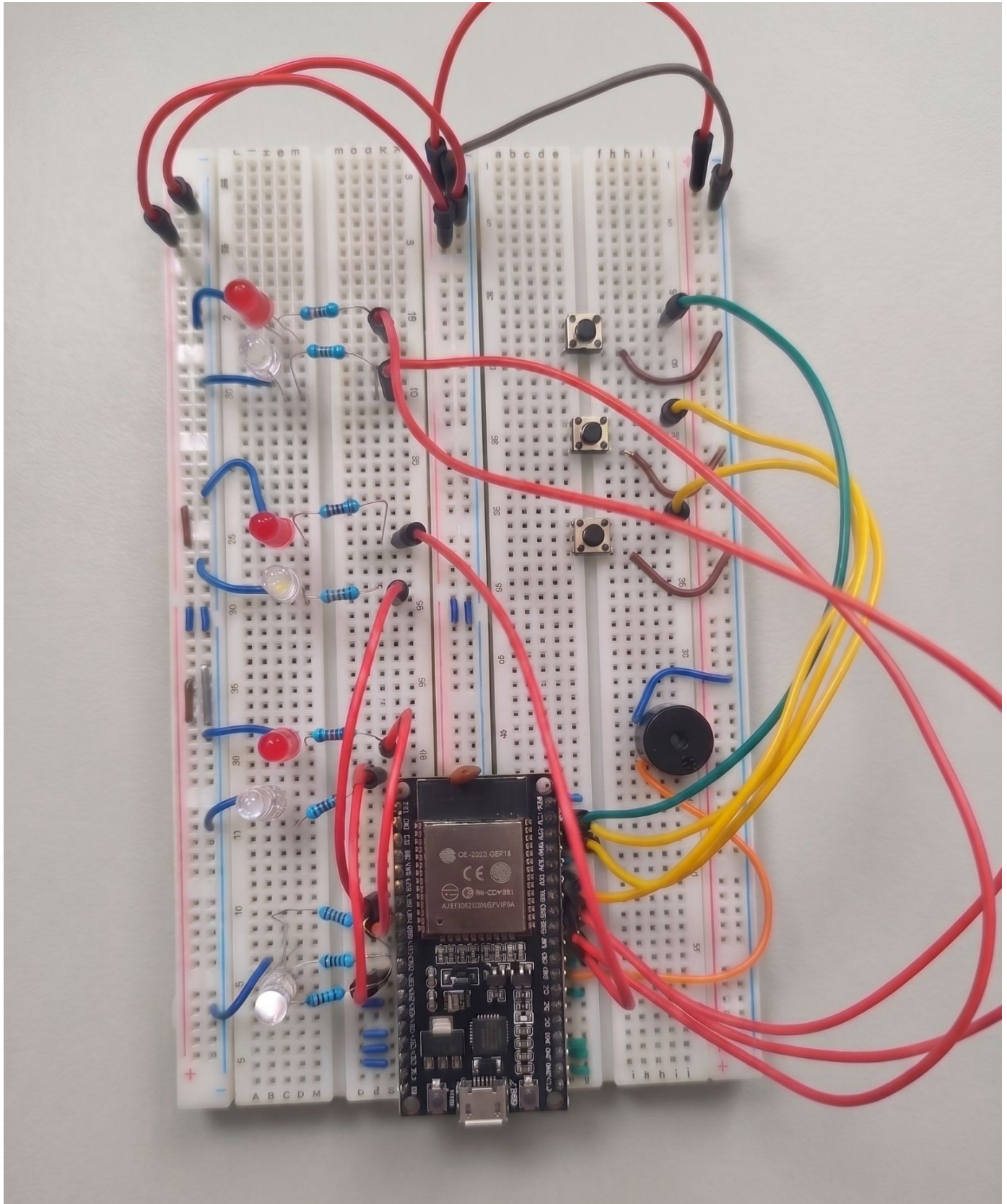
Per garantire la leggibilità del firmware e la manutenibilità del prototipo, è stata definita la seguente assegnazione dei pin:

TABELLA DI CABLAGGIO - MORSE ALARM

Componente	Pin ESP32 (GPIO)	Tipo Segnale	Funzione nel Codice
 1: Pulsante PUNTO	21	Digital Input	Input segnale breve Morse
 2: Pulsante LINEA	22	Digital Input	Input segnale lungo Morse
 3: Pulsante INVIO	23	Digital Input	Conferma della sequenza
 4: Buzzer Passivo	4	PWM / DAC	Emissione allarme e toni
 5: LED Rossi (x3)	33, 5, 19	Digital Output	Feedback input
 6: LED Bianchi (x3)	14, 32, 18	Digital Output	Convalida input
7: LED RGB (Rosso)	27	PWM / Output	Stato: Allarme / Errore
 8: LED RGB (Verde)	26	PWM / Output	Stato: Sistema OK / Successo
9: LED RGB (Blu)	25	PWM / Output	Stato: Wi-Fi / Connessione

3.2 Note sulla Realizzazione

- **Gestione del Debouncing:** Per i pulsanti è stato implementato un sistema di *debouncing* software per evitare che le vibrazioni meccaniche dei contatti vengano interpretate come molteplici pressioni, garantendo precisione nell'inserimento del codice Morse.
- **Isolamento delle Resistenze:** Le resistenze da **100 ohm** sono state posizionate sul lato anodo dei LED per limitare la corrente assorbita da ogni singolo ramo GPIO.



4. Logica di Controllo

Il firmware dell'ESP32 è strutturato come una **Macchina a Stati Finiti (FSM)**. Questa architettura garantisce la massima reattività del sistema, permettendo di gestire in parallelo e senza blocchi:

- Il server web in background.
- Il monitoraggio continuo dell'orario.
- La lettura asincrona degli input fisici (pulsanti).

4.1 Definizione degli Stati del Sistema

Stato 0: INIT_SYSTEM: Configura l'hardware, tenta la connessione Wi-Fi e sincronizza l'orario (NTP). Il LED RGB fornisce feedback visivo dello stato (Blu: connessione, Verde: successo, Rosso: errore). Avvia il server web locale.

Stato 1: IDLE: È la fase di riposo. Gestisce in background le richieste web (per impostare l'allarme da smartphone) e verifica continuamente se l'orario attuale coincide con la sveglia.

Stato 2: ALARM_TRIGGERED: Scatta l'allarme. Il sistema genera una sfida casuale a 3 bit (codice visivo) e la mostra accendendo/spegnendo di conseguenza i 3 LED Rossi.

Stato 3: WAITING_INPUT: Suona il buzzer. Legge in modo asincrono l'input dai pulsanti ("Punto" e "Linea"), componendo la stringa Morse e fornendo feedback acustico temporaneo alla pressione. In parallelo, un timer monitora il tempo trascorso: ogni 30 secondi di mancata risoluzione, il sistema incrementa il parametro PWM del buzzer, rendendo il suono progressivamente più intenso e fastidioso.

Stato 4: VALIDATION: Alla pressione di "Invio", verifica il codice. **Se errato:** segnala l'errore (RGB Rosso), applica una penalità incrementando ulteriormente l'intensità acustica del buzzer e obbliga a riprovare. **Se corretto:** accende un LED Bianco (barra di progresso visiva). Al completamento dei 3 step, passa alla disattivazione.

stato 5: ALARM_DISMISSED: Sfida superata. Spegne buzzer e LED. Applica un blocco software per impedire una nuova attivazione nello stesso minuto e ritorna allo stato IDLE.

4.2 Gestione Asincrona e Debouncing

Una sfida tecnica rilevante affrontata nello sviluppo del firmware è stata la lettura dei pulsanti meccanici. Per evitare falsi trigger o letture multiple indesiderate dovute al rimbalzo dei contatti fisici, è stata implementata una logica di blocco temporaneo (es. `while (digitalRead(PIN_PUNTO) == LOW); delay(150);`). Questa tecnica, pur essendo semi-bloccante, è stata calcolata in modo da non interferire con il duty cycle del suono dell'allarme, garantendo un feedback tattile e uditivo reattivo durante la digitazione veloce del codice Morse.

5. Analisi Energetica e Stima dei Consumi

Per valutare l'assorbimento di corrente del dispositivo "Morse Alarm", l'analisi teorica basata sui datasheet dei componenti è stata affiancata da misurazioni effettuate in laboratorio.

5.1 Assorbimento dei singoli componenti

- **Microcontrollore ESP32:** Il datasheet ufficiale indica una corrente operativa media di **80 mA**.
- **LED :** La corrente è stata ricavata misurando la caduta di tensione sulla resistenza di limitazione (100 Ohm):
 - **LED Rosso / RGB (Rosso):** 10,2 mA
 - **RGB (Verde):** 9,8 mA
 - **LED Bianco:** 4,1 mA
- **Buzzer Passivo :** Ponendo un amperometro in serie (con onda quadra di test a 1000 Hz), si è rilevato un assorbimento medio di **0,36 mA**, con un impatto energetico quasi trascurabile.

Possiamo suddividere i consumi di "Morse Alarm" in due scenari:

Scenario A: Standby Sistema in attesa. Wi-Fi e Web Server sono attivi, ma tutti i componenti ottici e acustici sono spenti per risparmio energetico e per non disturbare.

- ESP32: 80 mA
- LED e Buzzer: 0 mA
- **Assorbimento Totale:** ~80 mA
- **Potenza Continua:** $5V \times 0,080 A = 0,40 W$

Scenario B: Allarme (Picco transitorio massimo) Valuta l'istante assoluto di massimo assorbimento: allarme in corso, sfida massima attivata (3 LED Rossi), 2 livelli superati (2 LED Bianchi) e inserimento di un codice errato che attiva simultaneamente il buzzer e il flash RGB rosso.

- ESP32 : 80 mA
- 3x LED Rossi: 30,6 mA
- 2x LED Bianchi: 8,2 mA
- LED RGB (Rosso - errore): 10,2 mA
- Buzzer : 0,36 mA
- **Assorbimento di Picco :** ~129,36 mA
- **Potenza Massima di Picco:** $5V \times 0,129 A = 0,645 W$

Componente	Corrente Assorbita (mA)
ESP32	80,0
Buzzer passivo	0,36
Led Rosso	10,2
Led bianco	4,1
Led Rgb (Rosso)	10,2

5.2 Conclusioni sulla Sostenibilità Energetica

Il circuito risulta altamente efficiente, con picchi transitori inferiori a 1 Watt.

Tuttavia, alimentando il sistema con un comune powerbank da 10.000 mAh a fronte di un consumo continuo in Standby di 0,40 W, l'autonomia massima sarebbe di circa 78 ore (poco più di 3 giorni).

Trattandosi di un dispositivo con finalità di sveglia quotidiana, l'alimentazione "always-on" tramite comune alimentatore USB da parete rimane al momento la soluzione ottimale per garantire stabilità e continuità operativa a lungo termine.

Tuttavia, nell'ottica di un'ulteriore ingegnerizzazione del prodotto e per un'eventuale futura transizione verso un dispositivo "stand-alone" a batteria, si propongono le seguenti migliorie progettuali per abbattere ulteriormente i consumi:

- **Sostituzione Elettro-Ottica (LED Blu):** Si potrebbe optare per la sostituzione dei LED rossi della sfida visiva con dei LED blu, visto che il loro consumo è nettamente inferiore $I = 4,5 \text{ mA}$

6. Sviluppi e Implementazioni Future

Sebbene l'attuale implementazione confermi la solidità dell'interazione hardware/software, il passaggio del 'Morse Alarm' da prototipo dimostrativo a dispositivo embedded definitivo richiede ulteriori interventi. Di seguito si delineano le principali aree di sviluppo futuro:

- **Integrazione Hardware RTC (Real-Time Clock):** Attualmente il sistema dipende totalmente dalla connettività Wi-Fi e dai server NTP per conoscere l'orario. L'aggiunta di un modulo RTC dedicato permetterebbe al dispositivo di mantenere l'ora esatta e far scattare l'allarme anche in caso di assenza temporanea di rete domestica o blackout del router.
- **Espansione dell'Interfaccia Web:** A livello firmware, il web server locale potrà essere ampliato per permettere all'utente non solo di impostare l'orario, ma anche di modulare la difficoltà (es. passando da una sfida a 3 bit a una a 4 o 5 bit) e selezionare le frequenze base del segnale PWM per personalizzare il tono del buzzer.
- **Progettazione su PCB Dedicato:** Per aumentare la robustezza meccanica e ridurre le interferenze o falsi contatti tipici della breadboard, il prossimo step prevede il routing dello schematico su un Circuito Stampato (PCB) progettato ad hoc, ottimizzando gli ingombri del dispositivo finale.

CODICE SORGENTE:

<https://github.com/giovagio04/Morse-Alarm-ESP32>

